01211373 Machine Learning and Programming for Industry

Dimensionality Reduction & Clustering

Week 5 — Classical Machine Learning

scikit-learn • Ch. 5 & 10

Last Week → This Week

WEEK 4 RECAP

- Bagging reduces variance; boosting reduces bias
- Random forest, gradient boosting, XGBoost — all tree ensembles
- Feature importance turns a model into root-cause insight
- Every model this month has needed labels to learn from

TODAY

- What to do when you don't have labels at all
- PCA: compressing many features into a few
- k-means, hierarchical, and DBSCAN clustering
- Using DBSCAN to flag anomalies with zero labels

Today's Running Example: Unlabeled Vibration Spectra

A vibration sensor streams a 20-value frequency spectrum every cycle. Nobody has labeled which readings are normal or anomalous.

Find Structure

Group similar spectra together — do they naturally fall into “idle,” “normal,” “heavy load” without ever being told so?

Flag the Unusual

Some readings won't fit any group. Can we flag those as possible anomalies, with no labeled examples of failure to learn from?

Both are unsupervised: the algorithm never sees a single “correct” label.

From Supervised to Unsupervised

WEEKS 2–4

Supervised Learning

“Is this bearing reading Normal or Faulty?” — you know the answer for every training example.

Needs: features AND labels

WEEK 5

Unsupervised Learning

“Do these readings form natural groups, and which ones don't belong to any?” — no answer key exists.

Needs: features only

Why Reduce Dimensions First?

Our vibration spectrum has 20 frequency-bin features. Clustering directly in 20 dimensions is harder than it sounds.

Distances Blur Together

In high dimensions, the closest and farthest points start to look almost equally distant — clustering relies on distance actually meaning something.

Impossible to Visualize

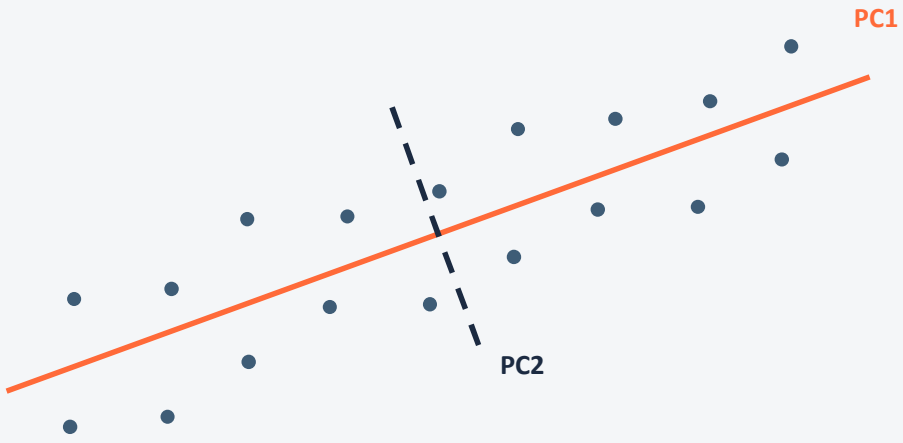
You can plot 2 or 3 dimensions on a screen. You cannot plot 20 — and you can't sanity-check what you can't see.

Mostly Redundant Anyway

Nearby frequency bins tend to rise and fall together — the real information often lives in just a handful of underlying directions.

PCA: Finding the Directions of Maximum Variance

PCA finds new axes — principal components — ranked by how much of the data's spread they capture. Keep the top few, drop the rest.



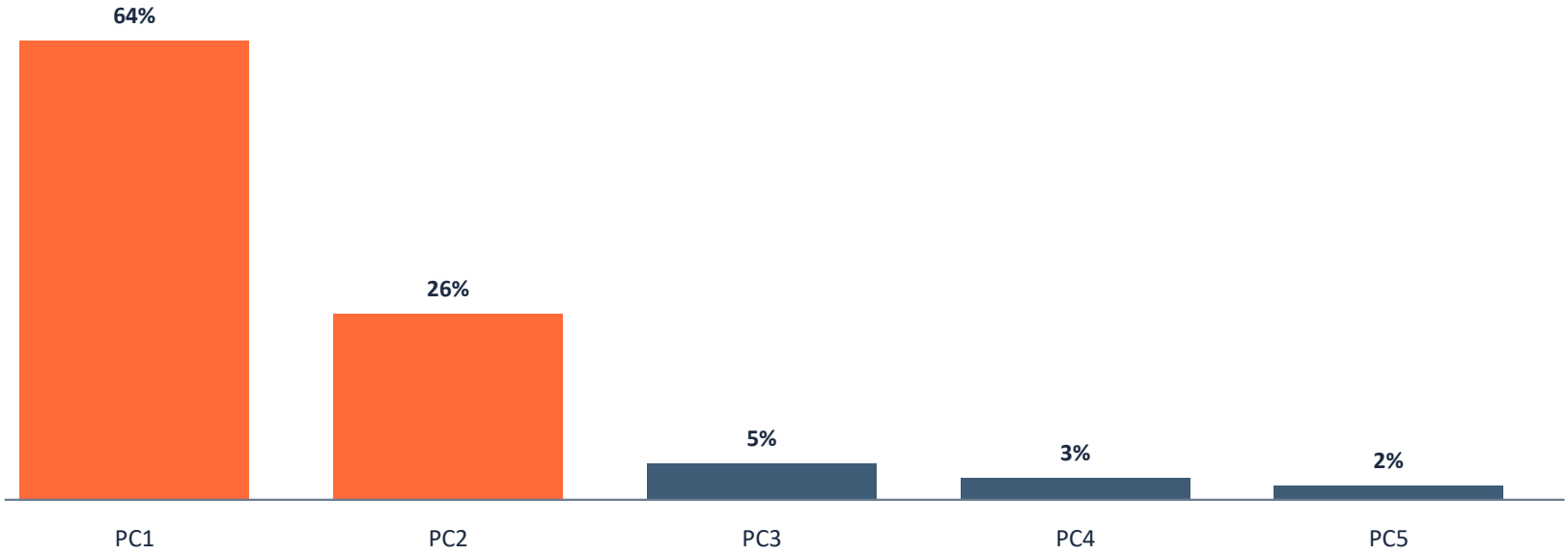
PC1 captures the most spread in the data.

PC2 is perpendicular to PC1, capturing the next most.

Both are combinations of ALL original features — not a subset of them.

PCA in Practice: How Many Components?

Each component comes with an explained variance ratio — how much of the total spread it accounts for. Plot them and look for a drop-off (a “scree plot”).



Illustrative example: the first two components already capture ~90% of the variance — a strong case for reducing to 2D.

A Brief Word on LDA & t-SNE

Two other dimensionality-reduction tools you'll see referenced elsewhere — worth recognizing, not worth a full lab.

LDA

Linear Discriminant Analysis — like PCA, but uses class labels to pick axes that best separate known categories. Needs labels, so it doesn't apply to today's unlabeled problem.

t-SNE

Excellent for visualizing high-dimensional clusters in 2D, but distorts distances — don't feed its output into a clustering algorithm the way we will with PCA.

Clustering: k-Means

Pick k centroids, assign every point to its nearest one, move each centroid to the mean of its assigned points, and repeat until stable.

You must choose k up front.

Assumes round, similarly-sized clusters.

Every point joins a cluster — there's no “none of the above.”

Clustering: Hierarchical (Briefly)

Merge the two closest points/clusters, repeat, and you get a full tree of groupings — a dendrogram — without ever choosing k.

Useful when: you want to explore groupings at several resolutions at once ("cut" the dendrogram at different heights), or don't want to commit to a fixed k.

Less common in production: $O(n^2)$ or worse, which gets slow fast on large sensor logs — k-means or DBSCAN are the more typical day-to-day choices.

```
from scipy.cluster.hierarchy import dendrogram, linkage   # good to recognize
```

Clustering: DBSCAN — Density-Based, Handles Noise

Groups points that are densely packed together; points that sit in a low-density region get labeled noise, not forced into a cluster.

No need to choose k.

Finds arbitrarily shaped clusters, not just round ones.

Labels sparse points −1 — exactly the anomaly flag we want.

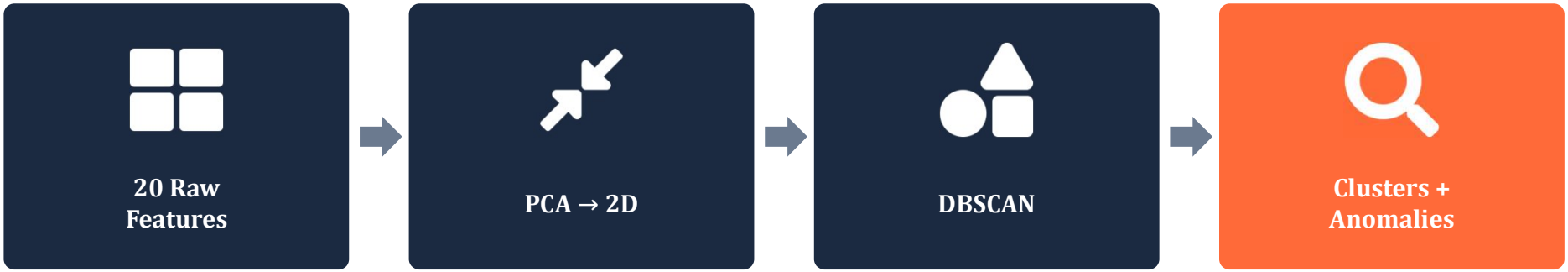
Choosing a Clustering Method

	k-Means	Hierarchical	DBSCAN
Choose k upfront?	Yes	No (cut tree later)	No
Cluster shape	Round / convex	Any (depends on linkage)	Any
Flags outliers as noise?	No	No	Yes
Scales to large data?	Yes	Poorly ($O(n^2)$+)	Yes

For today's anomaly-flagging task, DBSCAN's “noise” label is the whole point — that's why it anchors the lab.

Putting It Together: An Anomaly-Detection Pipeline

Neither step alone tells the full story — PCA makes the data visualizable and less noisy; clustering is what actually flags anomalies.



Today's Lab: Cluster & Flag, With No Labels

1	**Generate 20-dim vibration spectra**	Three operating states, plus scattered anomalies.
2	**Reduce with PCA**	20 features → 2 components; check explained variance.
3	**Cluster with k-means**	See it cleanly find the 3 states — and swallow every anomaly.
4	**Cluster with DBSCAN**	Flag the anomalies as noise, and check your work.

Key Takeaways

- PCA compresses many correlated features into a few informative ones.
- k-means forces every point into a cluster; DBSCAN doesn't have to.
- That one difference is what makes DBSCAN a genuine anomaly detector.

NEXT WEEK

Model Evaluation & Hyperparameter Tuning

Cross-validation, grid search, and pipelines — tying the whole semester's classical ML together.